

IE6-DPL: REPORT LAB 03

FIR FILTER IMPLEMENTATION IN MATLAB AND IN C

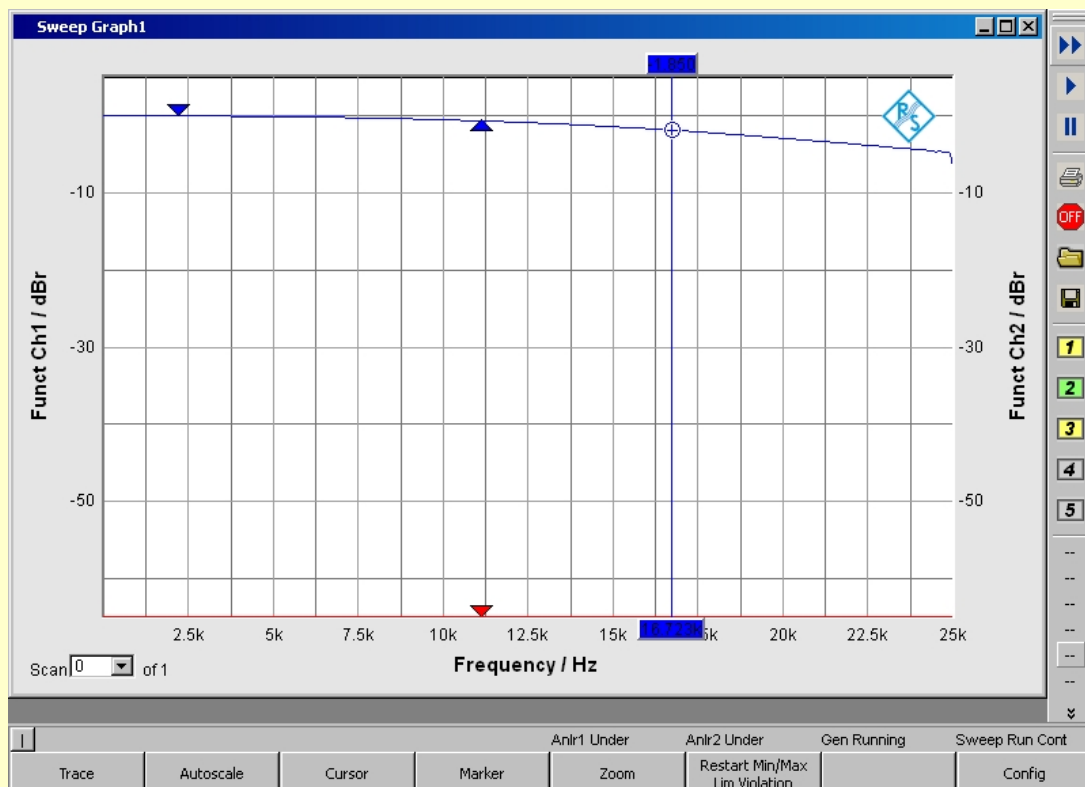
Team (Group number + letter as chosen in Moodle)	Team members	
	1	Efe Kardes
	2	Emir Avni Sahin
	3	

Please fill out the document electronically – including the yellow fields above – and upload it to Moodle once per team at the latest 30 min after the end of the lab session (no matter who uploads it, it will automatically be assigned to all members). Please note that the complete assignment can be found in another document.

Lab task 1: Amplitude response

1. Use again the starter project of the 1st lab and rename the file *main_adda_simple_Lab.c* to *fir_Lab.c*.
2. Measure the amplitude response of the board.
Briefly explain what you observe.

- Amplitude response of the board (Screenshot of UPV audio analyzer).



- Explain:

We can observe the baseline frequency response of the UniDAQ2 hardware without any additional digital FIR filter applied. The response is not perfectly flat and shows a slight low-pass characteristic. The amplitude slowly decreases at higher frequencies, for example around -1.8 dB at 16.723 kHz, and drops more strongly close to the Nyquist frequency. This behaviour is caused by the non-ideal ADC/DAC hardware response and the analog/digital filtering stages of the board.

Lab task 2: realtime implementation

1. Integrate the header files containing the FIR lowpass and highpass filter coefficients by using `#include "fir_LP.h"` and `#include "fir_HP.h"`. These two files have to be stored in the project directory.
2. Use ADC channel 0 as input and implement the high pass filter first, use DAC channel 0 as output.
3. Verify the functionality of the filter with the UPV.
4. Then add the low pass filter to the implementation. Use DAC channel 1 as output for this. Display both amplitude responses on the UPV, use both measurement channels for this.
5. Comment on the results and explain in particular the differences in stopband attenuation and passband ripple.

- Source code of low-pass and high-pass filter.

```
// --- HELPER FUNCTION: Q15 MAC Computation with Circular Buffer ---
```

```
static int16_t saturate_int16(int64_t x) {
```

```
    if (x > 32767) return 32767;
```

```
    if (x < -32768) return -32768;
```

```
    return (int16_t)x;
```

```
}
```

```
static int16_t fir_filter_q15(const int16_t *coeff, uint16_t n_coeff) {
```

```
    int64_t acc = 0;
```

```
    uint16_t i;
```

```
    uint16_t delay_index = g_u16FirIndex;
```

```

for (i = 0; i < n_coeff; i++) {

    acc += (int64_t)coeff[i] * (int64_t)g_i16FirDelay[delay_index];

    if (delay_index == 0)

        delay_index = n_coeff - 1;

    else

        delay_index--;

}

// Scale back to 16-bit (Q15 format)

acc = acc >> 15;

return saturate_int16(acc);

}

// --- ADC INTERRUPT: Shared Delay Line & Simultaneous Filtering ---

__interrupt void adcInt (void) {

    int16_t adc0 = PRU_addaRegs->adc[0];

    // Store newest input sample in circular delay line

    g_i16FirDelay[g_u16FirIndex] = adc0;

    // Calculate both FIR filters from the same ADC0 input

    g_i16HpOut = fir_filter_q15(asFirCoeHP, N_FIR_HP_COE);

    g_i16LpOut = fir_filter_q15(asFirCoeLP, N_FIR_LP_COE);

    // Update circular buffer index

    g_u16FirIndex++;

    if (g_u16FirIndex >= N_FIR_HP_COE)

        g_u16FirIndex = 0;

```

```

}

// --- DAC INTERRUPT: Hardware Output ---

__interrupt void dacInt (void) {

    // DAC 0 = high-pass output, DAC 1 = low-pass output

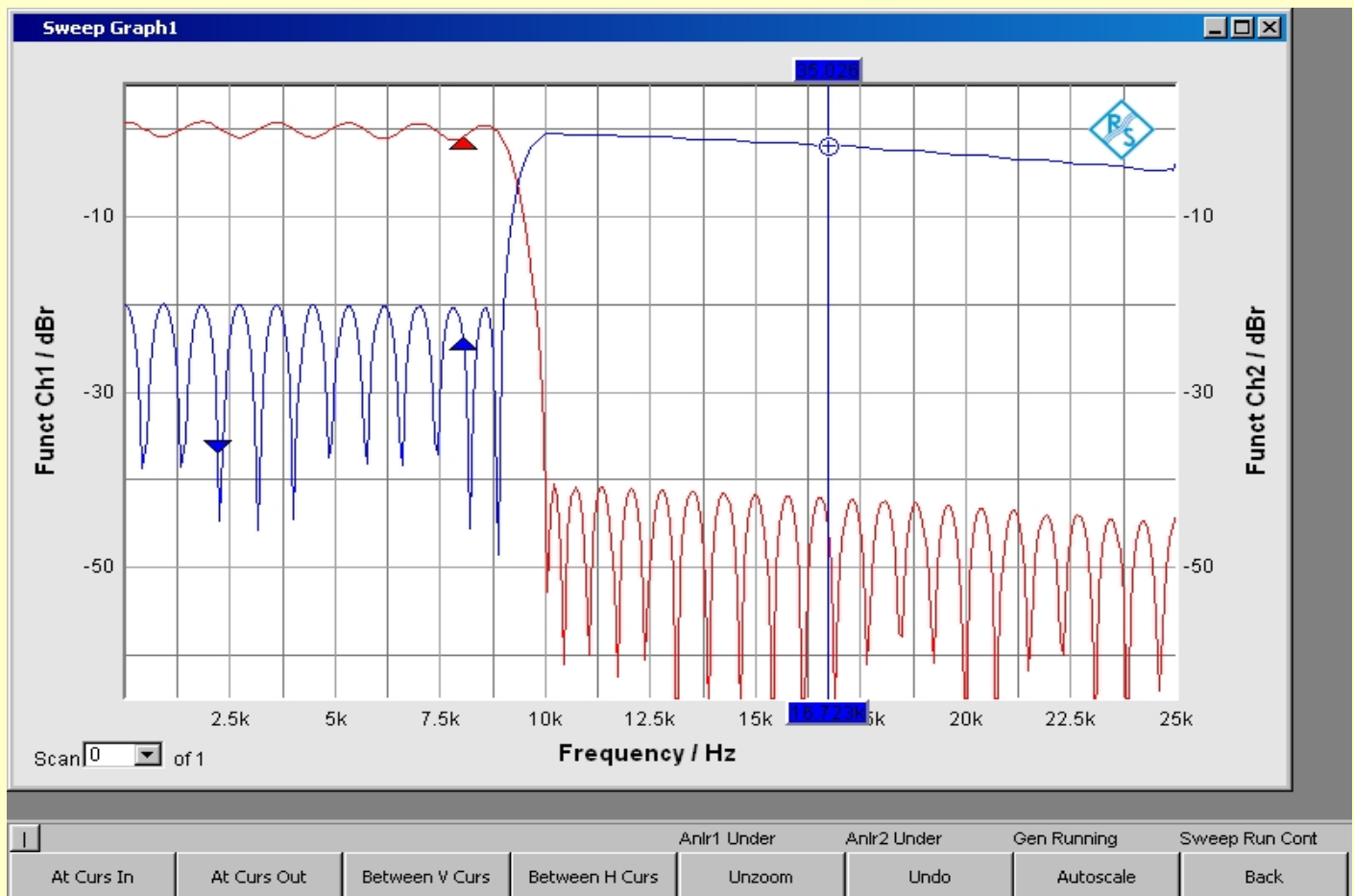
    PRU_addaRegs->dac[0] = g_i16HpOut;

    PRU_addaRegs->dac[1] = g_i16LpOut;

}

```

- Amplitude response of both filters measured with UPV audio analyzer.



- Your comments. (part 5)

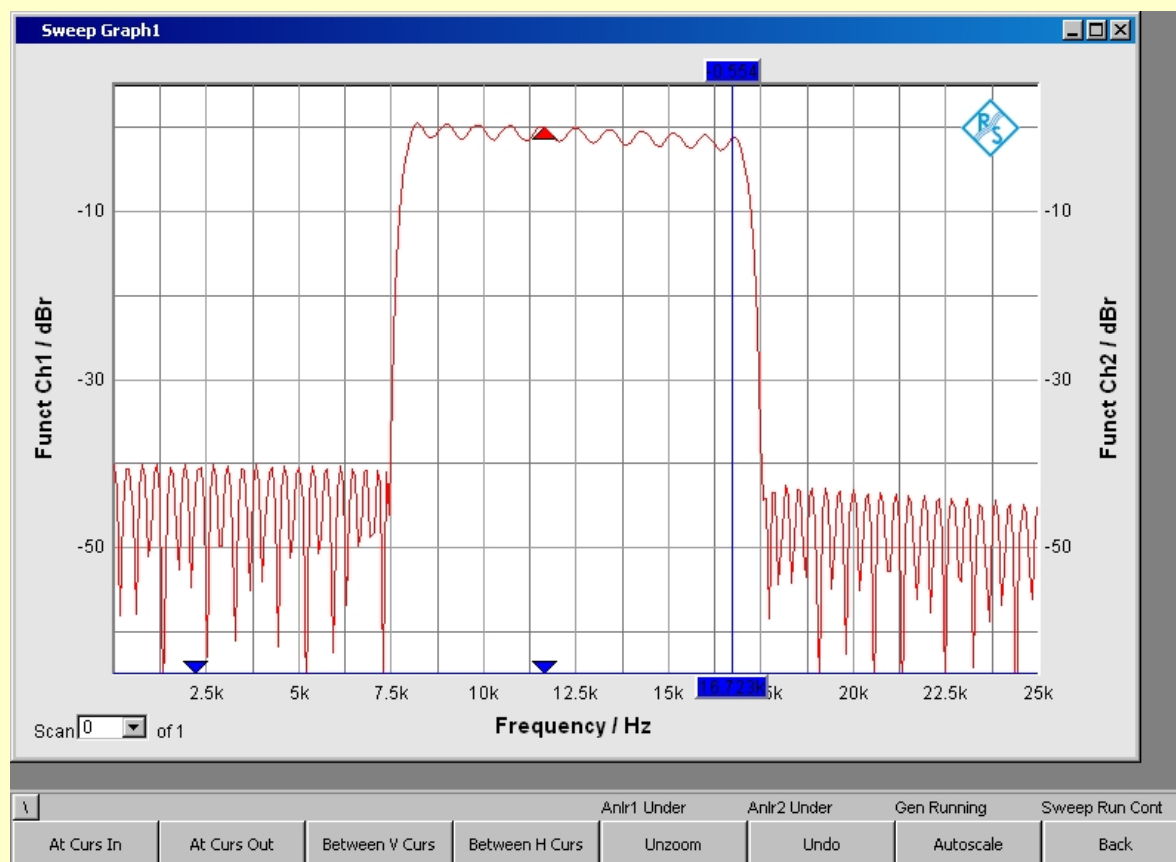
Our UPV measurements confirm the simultaneous operation of both FIR filters using the shared delay line. The trace from the lowpass (red) successfully meets the design specification, achieving a stopband attenuation of 40 dB. The trace from the highpass (blue) achieves a stopband attenuation of exactly 20 dB. The difference in stopband attenuation is caused by the different FIR coefficient design and the finite coefficient length. The low-pass filter has stronger stopband

suppression, while the high-pass filter shows more remaining ripple in the stopband.

Lab task 3: Bandstop and bandpass

1. Implement the bandstop and display the amplitude response on the UPV.
2. Extend the program for a bandpass: In a branch, the calculated product must now be subtracted for every second coefficient. Ensure that each coefficient is only multiplied once.
3. Display the bandstop and bandpass together on the UPV.
4. Why do the cut-off frequencies of the bandpass not correspond to those of the band-stop?

- Amplitude response bandpass



- C-code: bandpass and bandstop:

```
// --- HELPER FUNCTION: Simultaneous Bandstop and Bandpass ---
```

```
static void fir_bandstop_bandpass_q15(int16_t *pBandStop, int16_t *pBandPass)
```

```
{
```

```
    int64_t acc_bs = 0;
```

```
    int64_t acc_bp = 0;
```

```

uint16_t i;

uint16_t delay_index = g_u16FirIndex;

for (i = 0; i < N_FIR_LP_COE; i++)
{
    /*
     * Zero-inserted bandstop: actual 129-tap sequence uses only
     * samples x[n], x[n-2], x[n-4]...
     * Product is calculated once and reused for BS and BP to save cycles.
     */
    Int32_t product = (int32_t)asFirCoeLP[i] * (int32_t)g_i16FirDelay[delay_index];

    /* Bandstop: add all products. */
    acc_bs += product;

    /* Bandpass: subtract every second coefficient/product. */
    if (i & 1)
        acc_bp -= product;
    else
        acc_bp += product;

    /* Move two samples back in the circular delay line. */
    if (delay_index >= 2)
        delay_index -= 2;
    else
        delay_index += (N_FIR_BS_BP_COE - 2);
}

```

```

*pBandStop = saturate_int16(acc_bs >> FIR_Q_SHIFT);

```

```

    *pBandPass = saturate_int16(acc_bp >> FIR_Q_SHIFT);
}

// --- ADC INTERRUPT: Execution ---

__interrupt void adcInt (void)
{
    int16_t adc0 = PRU_addaRegs->adc[0];

    int16_t bandstop, bandpass;

    /* Store newest sample in the circular buffer. */

    g_i16FirDelay[g_u16FirIndex] = adc0;

    /* Calculate Task 3 filters from the same input. */

    fir_bandstop_bandpass_q15(&bandstop, &bandpass);

    g_i16BandStopOut = bandstop;

    g_i16BandPassOut = bandpass;

    g_u16FirIndex++;

    if (g_u16FirIndex >= N_FIR_BS_BP_COE)
        g_u16FirIndex = 0;
}

// --- DAC INTERRUPT: Output Mapping ---

__interrupt void dacInt (void)
{
    /* DAC0 = bandstop, DAC1 = bandpass. */

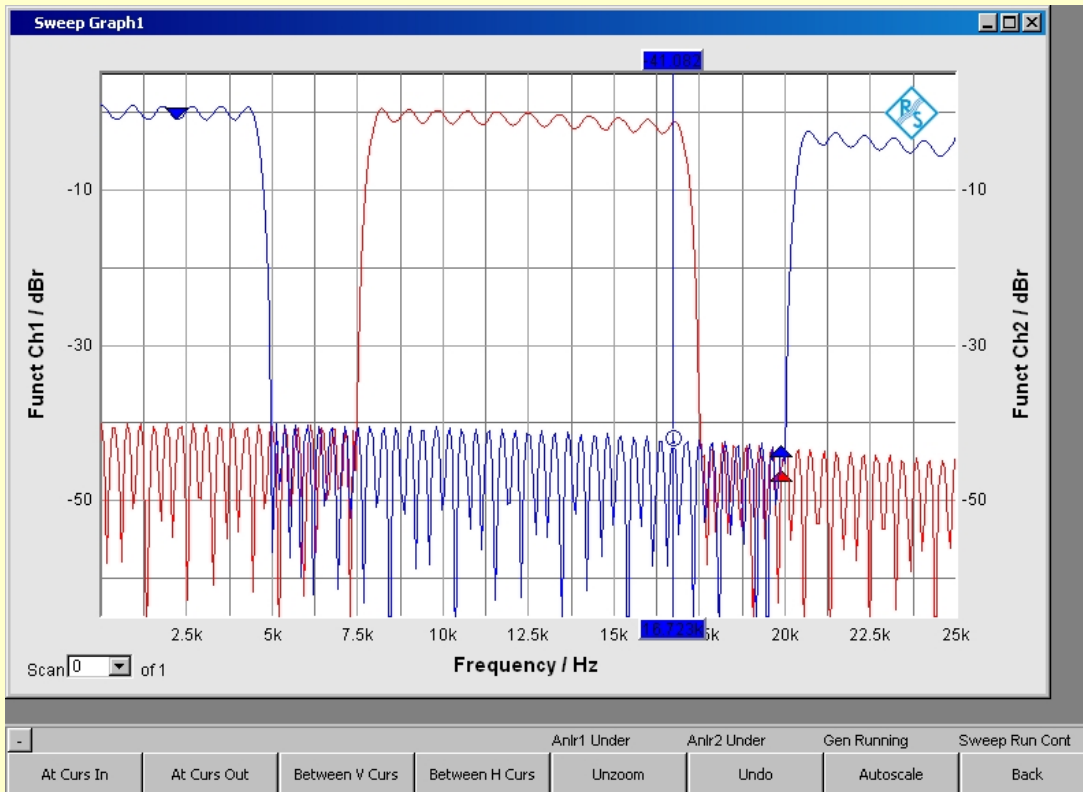
    PRU_addaRegs->dac[0] = g_i16BandStopOut;

```

```
PRU_addaRegs->dac[1] = g_i16BandPassOut;
```

```
}
```

- Amplitude responses bandpass and bandstop:



- Calculate the cut-off frequencies of the bandstop:
- The cut-off frequencies for the bandstop filter are: lower stopband ≈ 4.5 kHz, upper stopband ≈ 20 kHz.
- The cut-off frequencies of the Bandpass filter do not strictly align with the Bandstop filter. Modifying the coefficients to create the Bandpass filter resulted in a frequency shift, pulling the passband boundaries inward to approx. 7.9 kHz and 17.0 kHz, effectively narrowing the operational band compared to the original notch.

Lab task 4: Code profiling

- First calculate how many clock cycles are available in one sampling period. (The TMS320C6747 is clocked with 456 MHz.)
- Measure the number of clock cycles needed to compute two filters by profiling. A short instruction can be found in *Getting Started* [1].
- Determine (only theoretically!) the maximum sampling rate that would be possible to still calculate the two filters correctly.

- | |
|--|
| <ul style="list-style-type: none">• Clock cycles in one sampling period: $456.000.000 / 50.000 = 9120$ cycles |
| <ul style="list-style-type: none">• Clock cycles for filter calculation: maxFilterCycles: 3342, minFilterCycles: 3311 |
| <ul style="list-style-type: none">• Max. sampling frequency (rough estimation): $456.000.000 / 3342 = 136,445 \text{ Hz} \approx 136.4 \text{ kHz}$ |